Yaqeen Marketplace

Complete Code & Flow Explanation — Viva Preparation Guide

A line-by-line walkthrough of the full-stack React + PHP + MySQL project:
how every piece connects, what feeds what, and what each part of the code does.

React (Vite) | React Router | Context API | react-hook-form + Zod | Axios |
PHP | MySQL | Bootstrap

1. What is this project? (plain English)

Yaqeen is an online **marketplace** — a website where people buy and sell things (like a small OLX / Daraz). It has a **public side** (home page, product list, product details, about, contact, login, register) and an **admin side** (a dashboard where an administrator manages products, users and requests).

The project has **three layers**. Understanding these three layers is the single most important thing for your viva, because every feature is just these three layers talking to each other:

- **1. Frontend (React)** — everything you see in the browser. Buttons, forms, pages, tables. Written in JavaScript using React.
- **2. Backend (PHP)** — small programs on the server that receive requests, talk to the database, and send back answers. Written in PHP.
- **3. Database (MySQL)** — the permanent storage. Tables of rows: users, products, testimonials, etc.

A helpful analogy: the **Frontend is the waiter** (takes your order, shows you the menu), the **Backend (PHP) is the kitchen** (does the actual work), and the **Database is the fridge/store** (where all ingredients are kept). The waiter never cooks and never opens the fridge directly — it always asks the kitchen.

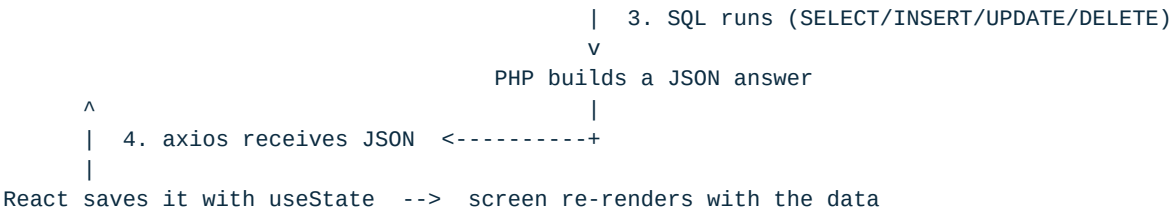
The technology stack (name each one confidently)

Tool	Layer	Its job in one line
React	Frontend	Builds the user interface out of reusable 'components'.
Vite	Frontend	The build tool: runs the dev server and bundles the final files.
React Router	Frontend	Shows different pages for different URLs, without reloading.
Context API	Frontend	Shares data (cart, login, theme) across many components.
react-hook-form	Frontend	Collects and manages form input easily.
Zod	Frontend	Checks that form input is valid (validation rules).
Axios	Frontend	Sends requests from React to the PHP backend over HTTP.
PHP	Backend	Receives requests, runs SQL, returns JSON.
MySQL	Database	Stores all data permanently in tables.
Bootstrap	Frontend	Ready-made CSS for a clean, responsive look.

2. The big picture — what feeds what

Here is the golden rule of the whole app, the flow you should be able to draw on a whiteboard:

```
BROWSER (React page)
      |  1. user does something (opens page / submits form)
      v
serviceApi.js  --- axios GET/POST --->  PHP file on server
                                            |  2. PHP reads request
                                            v
                                    connection.php  --> MySQL database
```



Every single feature in this project is that same loop. Registration, login, showing products, deleting a product — all of them follow these 4 steps. If you learn one feature end-to-end, you understand them all.

The two directions of data

- **Reading (GET)**: page opens → React asks PHP → PHP does **SELECT** → rows come back as JSON → React shows them. Example: the Products page.
- **Writing (POST)**: user submits a form → React sends the data → PHP does **INSERT / UPDATE / DELETE** → PHP replies success/fail → React shows a message. Example: Registration, Add Product, Delete.

3. File map — what each file is for

Frontend (src/)

File	What it does
main.jsx	The starting point. Wraps the app in all Context Providers and the Router.
router.jsx	The list of URLs -> which page component to show.
serviceApi.js	The bridge. All axios calls to PHP live here in one place.
context/AuthContext.jsx	Remembers who is logged in (user, login, logout).
context/CartContext.jsx	Holds the cart items (add / remove).
context/WishlistContext.jsx	Holds the wishlist items.
context/ThemeContext.jsx	Dark mode on/off.
components/Navigation.jsx	The top navbar (shown on every page).
components/Hero.jsx / Footer.jsx	The banner carousel and the page footer.
components/LandingPage.jsx	Home page: features, featured products, testimonials.
components/Login.jsx / Registration.jsx	The login and sign-up forms.
pages/Products.jsx	Public product list with search + filters.
pages/ProductDetail.jsx	One product's full page (uses id from the URL).
pages/About.jsx / Contact.jsx	About page (DB-driven) and contact form.
pages/Cart.jsx / Wishlist.jsx	Show cart / wishlist items.

pages/admin/Dashboard.jsx	Admin home: statistics + recent activity.
pages/admin/ProductsList.jsx	Admin product table (edit / delete).
pages/admin/ProductAdd.jsx / ProductEdit.jsx	Add / edit a product.
pages/admin/UsersList/UserAdd/UserEdit.jsx	Manage users.
pages/admin/Requests.jsx	Approve / reject requests.

Note: components/AdminLayout.jsx and MainLayout.jsx are layout components that exist in the project. The current router.jsx renders each admin page directly, so if asked, say the routes are defined flat in router.jsx.

Backend (backend/) — PHP files

File	What it does (SQL inside)
connection.php	Opens the connection to MySQL. Every other file includes it.
register_processing.php	INSERT a new user.
login_processing.php	SELECT a user by email+password to check login.
get_products.php	SELECT all products.
get_product.php	SELECT one product by id.
product_add_processing.php	INSERT a new product.
product_update_processing.php	UPDATE an existing product.
delete_product.php	DELETE a product by id.
get_users.php / get_user.php	SELECT all users / one user.
user_add_processing / user_update_processing / delete_user	INSERT / UPDATE / DELETE a user.
contact_processing.php	INSERT a contact message.
get_requests.php / update_request_status.php	SELECT requests / UPDATE a request status.
get_activities.php	SELECT dashboard activity rows.
get_team / get_milestones / get_values / get_stats.php	SELECT the About-page content.

4. Frontend, explained piece by piece

4.1 main.jsx — where the app starts

This is the **first file that runs**. It puts the whole app on the screen and wraps it in the four Context Providers plus the Router. Wrapping matters: anything *inside* a Provider can use that Provider's data.

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <ThemeProvider>
      <AuthProvider>
        <CartProvider>
          <WishlistProvider>
            <RouterProvider router={router} />   // <- all pages live here
          </WishlistProvider>
        </CartProvider>
      </AuthProvider>
    </ThemeProvider>
  </React.StrictMode>,
)
```

- **Line: createRoot(...).render(...)** — tells React to draw everything into the in index.html.
- **The nested Providers** — because RouterProvider is the innermost child, *every* page can read cart, wishlist, login and theme.
- **RouterProvider router={router}** — hands control to router.jsx, which decides which page to show.

4.2 router.jsx — URL to page mapping

React Router lets one website have many 'pages' without ever reloading the browser (this is called a **Single Page Application, SPA**). This file is simply a list: *for this URL, show this component*.

```
const router = createBrowserRouter([
  { path: "/",                    element: <LandingPage /> },
  { path: "/products",            element: <Products /> },
  { path: "/products/:id",        element: <ProductDetail /> },  // :id is a variable
  { path: "/login",               element: <Login /> },
  { path: "/register",            element: <Registration /> },
  { path: "/admin",               element: <Dashboard /> },
  { path: "/admin/products",      element: <ProductsList /> },
  { path: "/admin/products/edit/:id", element: <ProductEdit /> },
  ... (18 routes total)
]);
```

- **path** is the URL, **element** is the React component shown for it.
- **:id** is a **URL parameter** — a changing value. */products/3* means id = 3. The page reads it with useParams().
- Because it is an SPA, moving between pages needs the <Link> tag (not <a>), so the browser does not do a full reload.

4.3 Context — sharing data without 'prop drilling'

Problem: the navbar needs to know how many items are in the cart, but the cart is changed on a totally different page. Passing data through every component by hand is painful. **Context** is React's built-in solution: a shared box any component can read from. All four contexts follow the **exact same pattern**; here is Auth:

```
import { createContext, useState } from "react";
```

```
export const AuthContext = createContext();   // 1. create the box

function AuthProvider({ children }) {
  const [user, setUser] = useState("");        // 2. the shared value

  function login(name) { setUser(name); }      // 3. helpers to change it
  function logout() { setUser(""); }

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}                               // 4. wrap the app
    </AuthContext.Provider>
  );
}
```

- **createContext()** makes the shared box.
- **useState** holds the actual value (here the logged-in user's email).
- **value={{ user, login, logout }}** is exactly what other components receive.
- **{children}** means 'render whatever is placed inside me' — that is the whole app from main.jsx.

How a component uses it — Navigation.jsx reads four contexts at once with the **useContext** hook:

```
const { cartItems }        = useContext(CartContext);
const { wishItems }        = useContext(WishlistContext);
const { user, logout }     = useContext(AuthContext);
const { darkMode, toggleTheme } = useContext(ThemeContext);
...
<Link to='/cart'>Cart: {cartItems.length}</Link>   // live count in the navbar
{user ? <button onClick={logout}>Logout</button>
      : <Link to='/login'>Login</Link>}            // navbar changes when logged in
```

This is why the navbar instantly shows 'Cart: 1' after you add an item, and swaps Login for Logout after you sign in.

4.4 serviceApi.js — the ONE bridge to PHP

Every conversation with the backend goes through this file. Keeping it in one place means if the server address changes, you edit **one line**. That top line is exactly what we changed when deploying to InfinityFree.

```
const BASE_URL = "https://yaqeen.great-site.net/yaqeen-backend";

// WRITE example (send data in): POST
export const process_registration = async (data) => {
  const response = await axios.post(BASE_URL + "/register_processing.php", data);
  return response.data;
};

// READ example (get data out): GET
export const get_products = async () => {
  const response = await axios.get(BASE_URL + "/get_products.php");
  return response.data;
};
```

- **axios.post(url, data)** = send data TO the server (used by forms).
- **axios.get(url)** = ask the server FOR data (used to fill pages).
- **async / await** = the network takes time; 'await' politely waits for the reply before moving on.
- **response.data** = the JSON the PHP file printed with echo json_encode(...).

4.5 Forms — react-hook-form + Zod (Registration.jsx)

A form must (a) collect what the user types, (b) check it is valid, (c) send it. This project uses **react-hook-form** to collect and **Zod** to validate. Steps:

Step 1 — describe the rules with Zod (a 'schema')

```
const schema = z.object({
  fullName: z.string().min(3, { message: 'Minimum 3 characters.' }),
  email:    z.string().email({ message: 'Enter a valid email.' }),
  phone:    z.string().min(10, { message: 'Minimum 10 numbers.' }),
  password: z.string().min(6, { message: 'Minimum 6 characters.' }),
  ...
});
```

Plain English: fullName must be text of at least 3 letters; email must look like an email; and so on. If a rule fails, Zod produces the message shown to the user.

Step 2 — connect the schema to the form

```
const { register, handleSubmit, watch, formState: { errors } } =
        useForm({ resolver: zodResolver(schema) });
```

- **register('email')** — glue one input to the form (put on each <input>).
- **handleSubmit(submit)** — run validation first; only call our submit() if everything passes.
- **errors** — the object of failed-field messages, shown under each input.
- **watch('accountType')** — live-read a field; used to show Business fields only when 'seller' is chosen.

Step 3 — the input and its error message

```
<input {...register('email')} className='form-control' />
{errors.email && <p className='text-danger'>{errors.email.message}</p>}
```

Translation: *if errors.email exists, show its message in red*. This one line is the entire error-display pattern, repeated for every field.

Step 4 — send it to the backend

```
async function submit(data) {
  if (data.password !== data.confirmPassword) { alert('Passwords do not match!'); return; }
  if (!data.agreeTerms) { alert('You must agree to the terms.'); return; }
  try {
    const result = await process_registration(data);  // -> serviceApi -> PHP
    setServerMessage(result.message);                  // show PHP's reply
  } catch (error) {
    setServerMessage('A server error 500 occurred.');
  }
}
```

- **data** is the whole validated form as one object.
- **await process_registration(data)** is the moment React talks to PHP.
- **setServerMessage(...)** stores PHP's reply in state so it appears on screen.
- **try / catch** — if the server is unreachable, show a friendly error instead of crashing.

4.6 Login.jsx — form + context + redirect by role

Login is a form (same RHF+Zod pattern) plus two extra ideas: it saves the user into **AuthContext**, and it **redirects** using useNavigate depending on the role the database returns.

```
const navigate = useNavigate();
const { login } = useContext(AuthContext);

async function submit(data) {
```

```
  const result = await process_login(data);      // PHP checks email+password
  if (result.success) {
    login(data.email);                            // remember the user (context)
    if (result.role === 'Admin') navigate('/admin');  // admins -> dashboard
    else navigate('/');                           // normal users -> home
  } else {
    setServerMessage(result.message);             // 'Invalid email or password.'
  }
}
```

This is exactly what happened in the live demo: logging in as admin@yaqeen.pk sent us to /admin.

4.7 Showing data — useEffect + useState + map (Products.jsx)

This is the **read** pattern, used on almost every page. Three moves: make an empty box, fill it from the database when the page opens, then draw a card for each item.

```
const [productsData, setProductsData] = useState([]);   // 1. empty box

useEffect(() => {                                        // 2. runs once on open
  async function load() {
    const data = await get_products();                  //    ask PHP
    setProductsData(data);                              //    put rows in the box
  }
  load();
}, []);                                                 //    [] = only once
```

- **useState([])** — starts empty; if the fetch failed the page would just show nothing (not crash).
- **useEffect(fn, [])** — the empty [] means 'do this one time, right after the page appears'. This is where data loading belongs.
- **setProductsData(data)** — saving to state makes React **re-render** and the products appear.

Then the list is drawn with **.map()** — 'for each product, produce one card':

```
{filteredProducts.map((product) => (
  <div className='col-md-4' key={product.id}>
    <img src={product.image} />
    <h5>{product.title}</h5>
    <span>Rs. {Number(product.price).toLocaleString()}</span>
  </div>
))}
```

Filtering (search + category + price) is just JavaScript on the array before mapping — no server call needed:

```
const filteredProducts = productsData.filter((product) => {
  const matchesSearch = product.title.toLowerCase().includes((filters.search||'').toLowerCase());
  const matchesCategory = !filters.category || filters.category==='all' || product.category===filters.catego
  return matchesSearch && matchesCategory && ... ;
});
```

key={product.id} is required by React so it can tell list items apart efficiently. Always mention 'key' if asked about lists.

4.8 One item by its id — useParams (ProductDetail.jsx)

When you click a product, the URL becomes /products/3. This page reads that 3 and fetches only that product.

```
const { id } = useParams();                 // read :id from the URL (=3)
const [product, setProduct] = useState(null);

useEffect(() => {
  async function load() {
```

```
    const data = await get_product(id);     // GET get_product.php?id=3
    setProduct(data);
  }
  load();
}, [id]);                                   // re-run if the id changes

if (!product) return <p>Loading product...</p>;   // guard while waiting
```

- **useParams()** pulls variables out of the URL.
- **[id]** in useEffect — if the id changes, load the new product.
- **if (!product) return ...** — a 'guard': the first render has no data yet, so show 'Loading' to avoid an error.

4.9 Cart & Wishlist — pure Context (no database)

Add-to-cart does not touch the server; it just updates the Cart context array. That is why the navbar count changes instantly. ProductDetail calls the context helper:

```
const { addToCart } = useContext(CartContext);
function handleAddToCart() {
  addToCart(product);            // pushes product into cartItems[]
  alert(`Added ${product.title} to cart!`);
}
```

```
// inside CartContext.jsx
function addToCart(product) { setCartItems([...cartItems, product]); }
function removeFromCart(index) { setCartItems(cartItems.filter((item, i) => i !== index)); }
```

[...cartItems, product] = a NEW array with the old items plus the new one. React needs a new array to notice the change.

4.10 Admin CRUD — list, delete, and update

The admin pages are the same read pattern plus **write** actions. Deleting a product (ProductsList.jsx):

```
async function handleDelete(productId) {
  if (window.confirm('Are you sure...?')) {
    await process_product_delete(productId);              // 1. delete in the DB
    setProducts(products.filter(p => p.id !== productId)); // 2. remove from screen
    alert('Product deleted successfully!');
  }
}
```

Approving a request (Requests.jsx) — update the DB, then update the row locally so the badge flips to 'Approved':

```
async function handleApprove(requestId) {
  await process_request_status(requestId, 'Approved');    // UPDATE in DB
  setRequests(requests.map(req =>
    req.id === requestId ? { ...req, status: 'Approved' } : req));  // update screen
}
```

The Dashboard loads three things at once and then **calculates** the statistic numbers from the arrays — the numbers are never hard-coded:

```
const totalProducts = productsData.length;
const totalUsers    = usersData.length;
const totalSellers  = usersData.filter(u => u.role === 'Seller').length;
const pendingUsers  = usersData.filter(u => u.status === 'Pending').length;
```

5. Backend (PHP), explained piece by piece

Each PHP file is a tiny, self-contained program. They almost all share the **same 5-part skeleton**. Learn the skeleton once and every file becomes readable.

5.1 connection.php — open the database

Included by every other file. It opens one connection to MySQL. On the live server it holds the InfinityFree database details; on your laptop (XAMPP) it uses localhost/root.

```
<?php
  $connect = mysqli_connect(
    "sql211.infinityfree.com",   // host
    "if0_42345041",              // username
    "********",                  // password
    "if0_42345041_yaqeen"        // database name
  );
  if (!$connect) { die('Connection failed: ' . mysqli_connect_error()); }
?>
```

- **mysqli_connect(host, user, pass, dbname)** — the one function that logs PHP into MySQL.
- **$connect** — the open line; every query needs it.
- **die(...)** — if the connection fails, stop immediately with a clear message.

5.2 The shared skeleton (top of every endpoint)

```
header("Access-Control-Allow-Origin: *");                 // 1. allow the browser to call us
header("Access-Control-Allow-Methods: POST, GET, OPTIONS");
header("Access-Control-Allow-Headers: Content-Type");

if ($_SERVER['REQUEST_METHOD'] == 'OPTIONS') {            // 2. answer the browser pre-check
    http_response_code(200); exit();
}
include("connection.php");                                // 3. open the database

$rawData = file_get_contents("php://input");              // 4. read the JSON body axios sent
$data = json_decode($rawData);                            //    turn JSON text into a PHP object
```

- **CORS headers (1)** — the React app runs on a different address, so the browser blocks it unless PHP says 'you are allowed'. These headers are that permission.
- **OPTIONS pre-flight (2)** — before a POST, browsers send a tiny test request called OPTIONS; we just reply 200 OK and stop.
- **include connection.php (3)** — gives this file the $connect line to the database.
- **file_get_contents('php://input') (4)** — axios sends data as JSON in the request body; this reads that raw text, and json_decode turns it into $data with properties like $data->email.

GET files (get_products.php etc.) skip step 4 because they don't receive a body — they just SELECT and echo.

5.3 READ all — get_products.php (SELECT many)

```
$query  = "SELECT * FROM products";
$result = mysqli_query($connect, $query);

$products = [];
while ($row = mysqli_fetch_assoc($result)) {   // walk every row
    $products[] = $row;                        // add it to the array
}
echo json_encode($products);                   // print the array as JSON
```

- **mysqli_query($connect, $query)** — runs the SQL and returns a result set.
- **mysqli_fetch_assoc** in a while loop — pulls one row at a time as an array {column: value}.
- **json_encode(...)** — converts the PHP array into JSON text, which becomes response.data back in React.

5.4 READ one — get_product.php (SELECT by id)

```
$id = $_GET['id'];                                    // from URL ?id=3
$query = "SELECT * FROM products WHERE id = '$id'";
$result = mysqli_query($connect, $query);
$product = mysqli_fetch_assoc($result);               // just ONE row
echo json_encode($product);
```

$_GET['id'] reads the value after ?id= in the URL. One row -> one fetch_assoc (no loop).

5.5 CREATE — register_processing.php (INSERT)

```
$email    = $data->email;                // pull each field off the JSON object
$fullName = $data->fullName;
...
$query = "INSERT INTO users
          (account_type, full_name, email, phone, ...)
          VALUES
          ('$accountType', '$fullName', '$email', '$phone', ...)";

$process_query = mysqli_query($connect, $query);

if ($process_query) {
    echo json_encode(["success"=>true,  "message"=>"Account created for $fullName."]);
} else {
    echo json_encode(["success"=>false, "message"=>"...".mysqli_error($connect)]);
}
```

- **INSERT INTO ... VALUES ...** — the SQL that adds a new row.
- **if ($process_query)** — mysqli_query returns true on success; we reply with a success/failure JSON either way.
- **The reply shape** {success, message} is what React reads: result.success and result.message.

5.6 Login check — login_processing.php (SELECT to verify)

```
$query = "SELECT * FROM users WHERE email = '$email' AND user_password = '$password'";
$result = mysqli_query($connect, $query);

if (mysqli_num_rows($result) == 1) {                // exactly one match = correct login
    $user = mysqli_fetch_assoc($result);
    echo json_encode(["success"=>true, "role"=>$user["role"], "name"=>$user["full_name"]]);
} else {
    echo json_encode(["success"=>false, "message"=>"Invalid email or password."]);
}
```

mysqli_num_rows counts matches. 1 row = credentials correct; the returned 'role' is what Login.jsx uses to redirect admins.

5.7 UPDATE — update_request_status.php

```
$id = $data->id; $status = $data->status;
$query = "UPDATE requests SET status = '$status' WHERE id = '$id'";
mysqli_query($connect, $query);
```

5.8 DELETE — delete_product.php

```
$id = $data->id;
$query = "DELETE FROM products WHERE id = '$id'";
mysqli_query($connect, $query);
```

Notice the four SQL verbs map exactly to the four things a database can do — this is **CRUD**:

Word	SQL	Example file
Create	INSERT	register_processing.php, product_add_processing.php
Read	SELECT	get_products.php, get_product.php
Update	UPDATE	product_update_processing.php, update_request_status.php
Delete	DELETE	delete_product.php, delete_user.php

6. The database (MySQL) tables

All data lives in these tables. Each PHP file reads or writes exactly one of them.

Table	Holds	Filled by
users	Buyers, sellers, admin (login lives here)	register / admin add user
products	All marketplace products	admin Add Product
testimonials	Home-page reviews	seed data
contacts	Messages from the Contact form	contact form
requests	Approval requests for the admin	seed data
activities	Recent-activity rows on the dashboard	seed data
team_members / milestones / core_values / about_stats	About-page content	seed data

Admin login for the demo: admin@yaqeen.pk / admin123 (a row in the users table with role = 'Admin').

7. How it went live (InfinityFree)

- **npm run build** turned the React code into plain files inside a **dist/** folder (index.html + assets + images).
- Those files were uploaded by **FTP** into the host's **htdocs/** folder; the PHP files went into **htdocs/yaqeen-backend/**.
- A **.htaccess** file makes deep links (like /products) work on refresh by sending unknown paths back to index.html (needed for a Single Page App).
- The database was created in the InfinityFree panel and the tables imported; **connection.php** was filled with the live database details.

- One line in serviceApi.js (BASE_URL) was switched from localhost to the live domain so React talks to the live PHP.

8. Jargon buster (say these in your own words)

Term	Plain meaning
Component	A reusable piece of UI written as a function that returns HTML-like JSX.
JSX	The HTML-looking syntax inside React JavaScript.
State (useState)	A component's memory; changing it re-draws the screen.
Hook	A special React function starting with 'use' (useState, useEffect, useContext...).
useEffect	Run code at the right time — here, load data when a page opens.
Props	Inputs passed into a component (like function arguments).
Context	A shared value many components can read without passing props down.
Router / SPA	Show different pages for different URLs without a full reload.
Axios	The library that sends HTTP requests to the PHP backend.
API endpoint	One PHP URL that does one job (e.g. get_products.php).
JSON	A text format for data; both JS and PHP understand it.
CORS	Browser security; PHP headers grant the React app permission to call it.
CRUD	Create, Read, Update, Delete = INSERT, SELECT, UPDATE, DELETE.
mysqli	The PHP toolkit used to talk to MySQL.
Build (dist)	Turning source code into the final files the browser runs.

9. Likely viva questions & model answers

Q: Explain the overall architecture of your project.

A: It is a three-layer full-stack app. The **frontend** is React (the pages and forms). The **backend** is PHP (small files that run SQL). The **database** is MySQL. React talks to PHP using Axios over HTTP, PHP talks to MySQL using mysqli, and data travels back as JSON.

Q: What happens, step by step, when a user registers?

A: The Registration form collects input and Zod validates it. On submit, process_registration in serviceApi.js sends the data with axios.post to register_processing.php. That PHP file reads the JSON body, runs an INSERT into the users table, and echoes a JSON message. React stores that message in state and shows it on screen.

Q: How does data get from the database onto the Products page?

A: The page starts with an empty useState array. useEffect runs once when the page opens and calls get_products(), which does axios.get to get_products.php. PHP does SELECT * FROM products and returns the rows as JSON. React saves them with setProductsData, which re-renders, and .map() draws a card for each product.

Q: What is Context and why did you use it?

A: Context is React's way of sharing a value across many components without passing props through every level. I used it for the cart, wishlist, logged-in user, and dark mode — for example the navbar shows the live cart count and swaps Login for Logout, even though those values are changed on other pages.

Q: What do useState and useEffect do?

A: useState gives a component memory; when I update it the screen re-renders. useEffect runs code at a chosen time — I use it with an empty dependency array [] to load data from the backend exactly once when a page opens.

Q: What is the role of serviceApi.js?

A: It is the single bridge between React and PHP. Every axios call lives there, and the server address is one BASE_URL constant. When I deployed, I only had to change that one line from localhost to the live domain.

Q: How does login know whether to open the admin dashboard?

A: login_processing.php checks the email and password with a SELECT. If exactly one row matches, it returns that user's role. In Login.jsx, if result.role is 'Admin' I navigate('/admin'), otherwise navigate('/'). I also save the user in AuthContext.

Q: What are those header() lines and the OPTIONS check in every PHP file?

A: They are CORS headers. Because React runs on a different origin, the browser blocks the request unless the server permits it. The headers grant permission, and the OPTIONS block answers the browser's pre-flight test request with 200 OK.

Q: How is form validation done?

A: With Zod. I define a schema describing each field's rules (for example email must be a valid email, password at least 6 characters). react-hook-form runs the schema on submit via zodResolver and fills an errors object, which I display in red under each field.

Q: What does 'nothing is static' mean in your project?

A: Every list shown on the site is loaded from the MySQL database through PHP, not hard-coded in the JavaScript. Products, users, testimonials, requests, dashboard activity, and even the About page's team, milestones, values and stats all come from database tables.

Q: What is CRUD and where is it in your code?

A: CRUD is Create, Read, Update, Delete — the four SQL operations INSERT, SELECT, UPDATE, DELETE. Create is register / add product, Read is the get_* files, Update is edit product and approve request, Delete is delete product / user.

Q: Why Vite and what does 'build' produce?

A: Vite is the build tool and dev server. npm run build compiles all my React code into a small set of static files in a dist/ folder — one index.html plus bundled JS and CSS — which is what actually gets uploaded to the web host.

Q: How does clicking a product show its details?

A: The card is a Link to /products/:id. That URL has the product id. ProductDetail reads it with useParams(), calls get_product(id) which hits get_product.php?id=..., PHP does SELECT ... WHERE id = ..., and the single product is shown.

Q: If asked: is this code production-secure?

A: Honest answer: it is at course level. The SQL uses the values directly, so a real product would add prepared statements and password hashing. For this assignment the focus was the working full-stack data flow.

10. 60-second cheat sheet (memorise this)

- **main.jsx** starts the app and wraps it in 4 Providers + Router.
- **router.jsx** maps URLs to page components (18 routes).
- **serviceApi.js** = the only place axios talks to PHP; BASE_URL is the server address.
- **Context files** share cart / wishlist / user / theme across the app.
- **useState** = memory, **useEffect([])** = load on open, **.map()** = draw a list, **key** = list identity.
- **Zod schema** = validation rules; **register / handleSubmit / errors** = react-hook-form.
- **Every PHP file**: CORS headers → OPTIONS check → include connection.php → read JSON → run SQL → echo json_encode.
- **CRUD** = INSERT (create), SELECT (read), UPDATE (update), DELETE (delete).
- **Flow**: React → axios → PHP → MySQL → JSON → useState → screen.

Yaqeen Marketplace — full-stack React + PHP + MySQL. You built every layer; explain it as three layers talking to each other and you will do great.

11. Proof: every data block is dynamic

If sir says 'show me nothing is static', open these pages and point at where each list comes from the database. Every one starts as an empty useState and is filled by a get_ call in useEffect.

What you see on screen	Comes from table	Loaded by
Products (grid, detail, featured)	products	get_products / get_product
Home-page testimonials	testimonials	get_testimonials
Admin users table + stats	users	get_users
Requests table	requests	get_requests
Dashboard recent activity	activities	get_activities
Dashboard number cards	products+users	computed with .length / .filter
Dashboard System Status + storage	system_status	get_status
About: team, journey, values, stats	team_members, milestones, core_values, about_stats	get_team / get_milestones / get_values / get_stats
Contact info card	contact_info	get_contact_info

Honest note to remember: a few things are intentionally static *labels*, not data — page headings, the Hero carousel captions, the 'Why Choose Yaqeen' / 'How It Works' text, and the footer. These are wording, not records, so they live in the JSX (the same way your instructor's reference keeps them). If asked, say exactly that: *'all data is from the database; only fixed page labels are in the code.'*

12. Complete trigger index (nothing skipped)

Every 'thing that makes code run' in the app falls into 7 trigger types. Below is **every occurrence**, with the file, the trigger, and what it fires. If sir points at any feature, find its row.

Type A — page opens (useEffect with empty []) -> load data once

File	Fires	Shows
LandingPage.jsx:13	get_products + get_testimonials	featured products + testimonials
Products.jsx:15	get_products	product grid
Dashboard.jsx:11	get_products, get_users, get_activities, get_status	stats + activity + system status
ProductsList.jsx:12	get_products	admin product table
UsersList.jsx:12	get_users	admin user table

Requests.jsx:9	get_requests	requests table
About.jsx	get_team, get_milestones, get_values, get_stats	about sections
Contact.jsx	get_contact_info	contact info card

Type B — URL has an :id (useEffect with [id]) -> load one row, prefill form

File	Fires	Purpose
ProductDetail.jsx:17	get_product(id)	show one product
ProductEdit.jsx:40	get_product(id) then reset(...)	fill the edit form
UserEdit.jsx:36	get_user(id) then reset(...)	fill the edit form

reset(...) from react-hook-form is what drops the loaded values into every input box.

Type C — form submit (onSubmit = handleSubmit(submit)) -> write to DB

File	Calls	SQL result
Registration.jsx:72	process_registration	INSERT user
Login.jsx:63	process_login	SELECT check, then navigate by role
Contact.jsx:52	process_contact (then reset())	INSERT contact message
ProductAdd.jsx:54	process_product_add	INSERT product
ProductEdit.jsx:85	process_product_update({...data, id})	UPDATE product
UserAdd.jsx	process_user_add	INSERT user
UserEdit.jsx:78	process_user_update({...data, id})	UPDATE user

Type D — button click (onClick) -> action

File	Handler	What it does
ProductsList.jsx:126	handleDelete -> process_product_delete	DELETE product (with confirm)
UsersList.jsx:137	handleDelete -> process_user_delete	DELETE user (with confirm)
Requests.jsx:154	handleApprove -> process_request_status	UPDATE status = Approved
Requests.jsx:159	handleReject -> process_request_status	UPDATE status = Rejected

UsersList.jsx:120	handleStatusChange (dropdown)	screen only - does NOT save to DB
ProductDetail.jsx:101	handleAddToCart -> addToCart	adds to Cart context
ProductDetail.jsx:102	handleAddToWishlist -> addToWishlist	adds to Wishlist context
Cart.jsx:49 / Wishlist.jsx:31	removeFromCart / removeFromWishlist	removes from context list
Navigation.jsx:40 / 46	toggleTheme / logout	dark mode / clear user

Be honest about UsersList.jsx:120 - the status dropdown only updates the screen; it has no serviceApi call, so a refresh loses it. If sir asks, say 'that one is local-only, the delete and edit are the DB actions.'

Type E — live filter (register + watch, no server call)

File	Watches	Effect
Products.jsx:10	watch() (search, category, price)	filters the grid as you type
ProductsList.jsx:9	watch('searchTerm')	filters the product table
UsersList.jsx:9	watch('searchTerm')	filters the user table
Requests.jsx:18	watch('filterType')	filters requests by type

Type F — context read (useContext) -> shared state

File	Reads	Used for
Navigation.jsx:10-13	cart, wishlist, auth, theme	counts, Login/Logout swap, theme button
Cart.jsx:8	cartItems, removeFromCart	cart table + total
Wishlist.jsx:8	wishItems, removeFromWishlist	wishlist grid
ProductDetail.jsx:11-12	addToCart, addToWishlist	the two buttons
Login.jsx:13	login	remember the user after login

Type G — navigation (Link / useNavigate)

- **<Link to='...'>** — every menu item, product card, and admin sidebar link. Changes the URL without reloading.
- **navigate('/admin') or navigate('/')** — Login.jsx after a correct login, based on role.
- **navigate('/admin/products')** — the Cancel buttons on the add/edit forms.

13. Page-by-page flow (one line each)

Route / page	On open it...	You can...
/ LandingPage	loads products + testimonials	browse, go to a product, sign up
/products	loads all products	search / filter / open a product
/products/:id ProductDetail	loads that one product by id	add to cart / wishlist
/about	loads team, milestones, values, stats	read (all from DB)
/contact	loads contact info	submit the contact form (INSERT)
/cart	reads Cart context	remove items, see total
/wishlist	reads Wishlist context	remove items
/login	-	log in (SELECT check) -> redirect by role
/register	-	create account (INSERT user)
/admin Dashboard	loads products, users, activities, status	see stats, jump to actions
/admin/products	loads products	search, edit, delete
/admin/products/add	-	add product (INSERT)
/admin/products/edit/:id	loads product, fills form	update product (UPDATE)
/admin/users	loads users	search, edit, delete
/admin/users/add	-	add user (INSERT)
/admin/users/edit/:id	loads user, fills form	update user (UPDATE)
/admin/requests	loads requests	approve / reject (UPDATE)

14. Extra answers sir may want

Q: How does the Edit page already show the old values?

A: The route carries the id (e.g. /admin/products/edit/3). useEffect reads it with useParams, calls get_product(id), and then reset(...) from react-hook-form drops those values into every input. On submit I send {...data, id} so product_update_processing.php knows which row to UPDATE.

Q: What exactly triggers a database read versus a write?

A: A read is triggered by useEffect running on page open (or when an :id changes) and calls a get_ function. A write is triggered by a form submit (handleSubmit) or a button onClick, and calls a process_ function that POSTs to PHP.

Q: Name every place data is loaded from the database.

A: LandingPage, Products, ProductDetail, About, Contact, and the four admin pages Dashboard, ProductsList, UsersList and Requests - plus the two edit pages that prefill their forms. Each uses useEffect + a get_ function.

15. Call-and-response drill: point at the line

This is exactly how sir grills you: he names an operation, you explain a step, he says **'show me the line'**, you point, he says **'then?'**. Below is every operation as that back-and-forth. Answer **one hop per question, in order**, and never skip. If you blank on a line number, name the code itself ("the axios.post line in serviceApi").

The fixed 6-hop order for every operation: 1) trigger (useEffect / onClick / onSubmit) → 2) the get_ or process_ call → 3) axios in serviceApi.js → 4) the SQL word in the PHP → 5) json_encode back → 6) the setState that redraws.

READ - view all products (the classic)

Sir asks / you say	Point at this exact line	File:line
Explain view. What triggers it?	`useEffect(() => { load(); }, []);`	*Products.jsx: 15*
What fires the request?	`const data = await get_products();`	*Products.jsx: 17*
serviceApi line?	`await axios.get(BASE_URL + "/get_products.php");`	*serviceApi.js :77*
The PHP query?	`$query = "SELECT * FROM products";`	*get_product s.php:14*
How does it come back?	`echo json_encode($products);`	*get_product s.php:22*
Where is it stored?	`setProductsData(data);`	*Products.jsx: 18*
Where is it drawn?	`{filteredProducts.map((product) => ( ... ))}`	*Products.jsx: 99*

READ one - person CLICKS a product

Sir asks / you say	Point at this exact line	File:line
Person clicks a product - the line?	`<Link to={`/products/${product.id}`}>`	*Products.jsx:101*
Which route catches it?	`{ path: "/products/:id", element: <ProductDetail /> }`	*router.jsx:23*
How is the id read?	`const { id } = useParams();`	*ProductDetail.jsx:10*
What triggers the load?	`useEffect(() => { load(); }, [id]);`	*ProductDetail.jsx:17*
The request line?	`const data = await get_product(id);`	*ProductDetail.jsx:19*
serviceApi?	`await axios.get(BASE_URL + "/get_product.php?id=" + id);`	*serviceApi.js:83*
PHP?	`$id = $_GET['id']; ... WHERE id = '$id'`	*get_product.php:15,17*
Show on screen?	`setProduct(data); -> <h1>{product.title}</h1>`	*ProductDetail.jsx:20,68*

CREATE - add a product

Sir asks / you say	Point at this exact line	File:line
Person clicks Add Product - line?	`<button type="submit">Add Product</button>`	*ProductAdd.jsx:233*
What catches the submit?	`<form onSubmit={handleSubmit(submit)}>`	*ProductAdd.jsx:54*
What sends the data?	`const result = await process_product_add(data);`	*ProductAdd.jsx:38*
serviceApi?	`await axios.post(BASE_URL + "/product_add_processing.php", data);`	*serviceApi.js:45*
PHP reads the body?	`$data = json_decode(file_get_contents("php://input"));`	*product_add_processing.php:14*
The actual save?	`$query = "INSERT INTO products (...) VALUES (...)";`	*product_add_processing.php:31*
Runs the query?	`mysqli_query($connect, $query);`	*product_add_processing.php:36*
How does the user know?	`setServerMessage(result.message);`	*ProductAdd.jsx:39*

UPDATE - edit a product

Key idea: {...data, id} sends the id so PHP knows WHICH row to update.

Sir asks / you say	Point at this exact line	File:line
How does the form show old values?	`useEffect -> get_product(id) -> reset({...})`	*ProductEdit.jsx:40,42,44*
Clicks Update - what sends it?	`await process_product_update({ ...data, id });`	*ProductEdit.jsx:69*
serviceApi?	`await axios.post(BASE_URL + "/product_update_processing.php", data);`	*serviceApi.js:51*
The SQL?	`$query = "UPDATE products SET ... WHERE id = '$id'";`	*product_update_processing.php:30*
Feedback?	`setServerMessage(result.message);`	*ProductEdit.jsx:70*

DELETE - delete a product

Sir asks / you say	Point at this exact line	File:line
Clicks Delete - the line?	`<button onClick={() => handleDelete(product.id)}>`	*ProductsList.jsx:126*
What runs first?	`if (window.confirm('Are you sure...?')) {`	*ProductsList.jsx:21*
The server line?	`await process_product_delete(productId);`	*ProductsList.jsx:23*
serviceApi?	`await axios.post(BASE_URL + "/delete_product.php", { id });`	*serviceApi.js:63*
SQL?	`$query = "DELETE FROM products WHERE id = '$id'";`	*delete_product.php:21*
Remove from screen?	`setProducts(products.filter(p => p.id !== productId));`	*ProductsList.jsx:24*

Same pattern for the OTHER tables - here are their exact lines too (nothing skipped):

CREATE - register (public sign-up = new user)

Sir asks / you say	Point at this exact line	File:line
Submits the form?	`<form onSubmit={handleSubmit(submit)}>`	*Registration.jsx:72*
Sends it?	`const result = await process_registration(data);`	*Registration.jsx:52*
serviceApi?	`await axios.post(BASE_URL + "/register_processing.php", data);`	*serviceApi.js:15*
SQL?	`$query = "INSERT INTO users (...) VALUES (...)";`	*register_processing.php:34*
Feedback?	`setServerMessage(result.message);`	*Registration.jsx:53*

CREATE - contact message

Sir asks / you say	Point at this exact line	File:line
Submits the form?	`<form onSubmit={handleSubmit(submit)}>`	*Contact.jsx:62*
Sends it?	`const result = await process_contact(data);`	*Contact.jsx:41*
serviceApi?	`await axios.post(BASE_URL + "/contact_processing.php", data);`	*serviceApi.js:21*
SQL?	`$query = "INSERT INTO contacts (...) VALUES (...)";`	*contact_processing.php:28*

CREATE - admin adds a user

Sir asks / you say	Point at this exact line	File:line
Clicks Add User?	`<button type="submit">Add User</button>`	*UserAdd.jsx:183*
Catches submit?	`<form onSubmit={handleSubmit(submit)}>`	*UserAdd.jsx:51*
Sends it?	`const result = await process_user_add(data);`	*UserAdd.jsx:35*
serviceApi?	`await axios.post(BASE_URL + "/user_add_processing.php", data);`	*serviceApi.js:27*
SQL?	`$query = "INSERT INTO users (...) VALUES (...)";`	*user_add_processing.php:32*

UPDATE - admin edits a user

Sir asks / you say	Point at this exact line	File:line
Form fills how?	`useEffect -> get_user(id) -> reset({...})`	*UserEdit.jsx: 36,38,40*
Clicks Update - sends?	`await process_user_update({ ...data, id });`	*UserEdit.jsx: 62*
serviceApi?	`await axios.post(BASE_URL + "/user_update_processing.php", data);`	*serviceApi.js :33*
SQL?	`$query = "UPDATE users SET ... WHERE id = '$id'";`	*user_update_processing.php:32*

DELETE - admin deletes a user

Sir asks / you say	Point at this exact line	File:line
Clicks Delete - line?	`<button onClick={() => handleDelete(user.id)}>`	*UsersList.jsx :137*
Server line?	`await process_user_delete(userId);`	*UsersList.jsx :23*
serviceApi?	`await axios.post(BASE_URL + "/delete_user.php", { id });`	*serviceApi.js :57*
SQL?	`$query = "DELETE FROM users WHERE id = '$id'";`	*delete_user.php:21*
Remove from screen?	`setUsers(users.filter(u => u.id !== userId));`	*UsersList.jsx :24*

UPDATE - approve / reject a request

Sir asks / you say	Point at this exact line	File:line
Clicks Approve - line?	`<button onClick={() => handleApprove(request.id)}>`	*Requests.jsx :154*
What runs?	`async function handleApprove(requestId) {`	*Requests.jsx :20*
The server line?	`await process_request_status(requestId, 'Approved');`	*Requests.jsx :22*
(Reject is the twin)	`onClick handleReject -> process_request_status(id,'Rejected')`	*Requests.jsx :159,32*
serviceApi?	`await axios.post(BASE_URL + "/update_request_status.php", { id, status });`	*serviceApi.js :69*
SQL?	`$query = "UPDATE requests SET status = '$status' WHERE id = '$id'";`	*update_request_status.php:22*
Screen without reload?	`setRequests(requests.map(req => req.id===id ? {...req,status} : req))`	*Requests.jsx :23*

LOGIN - check credentials + redirect

Sir asks / you say	Point at this exact line	File:line
Clicks Log In?	`<form onSubmit={handleSubmit(submit)}>`	*Login.jsx:63*
The request?	`const result = await process_login(data);`	*Login.jsx:31*
serviceApi?	`await axios.post(BASE_URL + "/login_processing.php", data);`	*serviceApi.js:39*
PHP check?	`"SELECT * FROM users WHERE email='$email' AND user_password='$password'"`	*login_processing.php:23*
Correct = how does it know?	`if (mysqli_num_rows($result) == 1) {`	*login_processing.php:27*
Remember the user?	`login(data.email);`	*Login.jsx:34*
Send admin to dashboard?	`if (result.role === 'Admin') navigate('/admin');`	*Login.jsx:36*

The two connector lines he always asks for

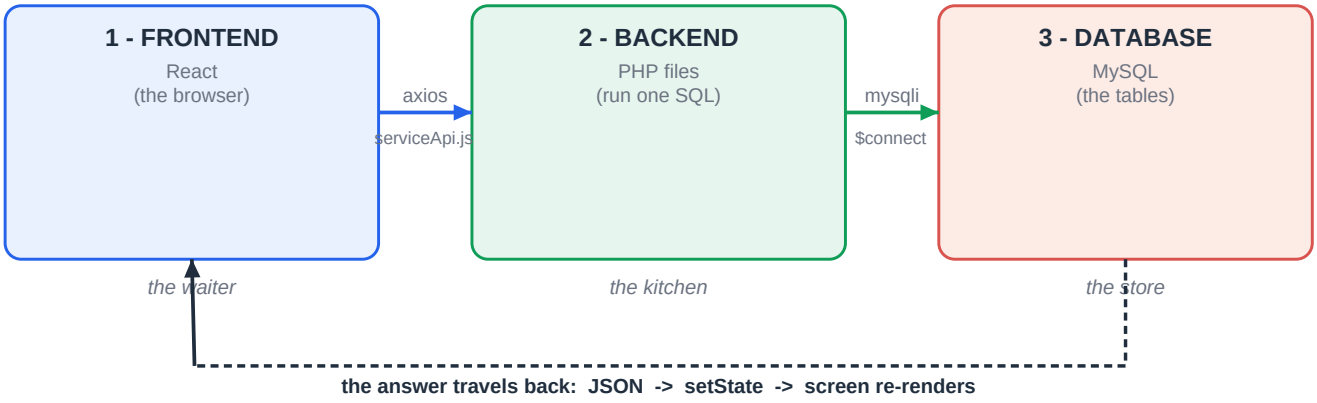
- **'What connects React and PHP?'** -> axios in serviceApi.js. Point at: `const BASE_URL = "...";` (serviceApi.js:9).
- **'What connects PHP and MySQL?'** -> `mysqli_query($connect, $query)`, and $connect is made by `mysqli_connect(...)` in connection.php.

If you remember only one thing: every operation is trigger -> get_/process_ -> axios -> SQL -> json_encode -> setState. Point at those six lines and you have answered him.

16. Picture it - diagrams to remember

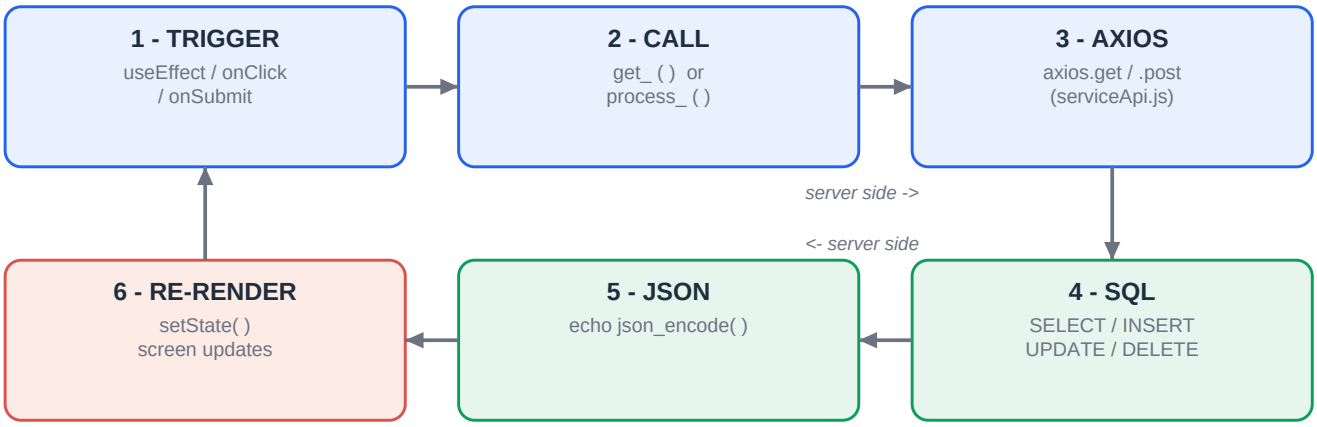
Same content as before, now as pictures. Memorise these four and the whole project fits in your head.

Diagram 1 - the whole app is 3 layers passing messages



React (waiter) never touches the database. It asks PHP (kitchen) via axios; PHP asks MySQL (store) via mysqli; the answer walks back as JSON.

Diagram 2 - the 6-hop loop behind EVERY feature



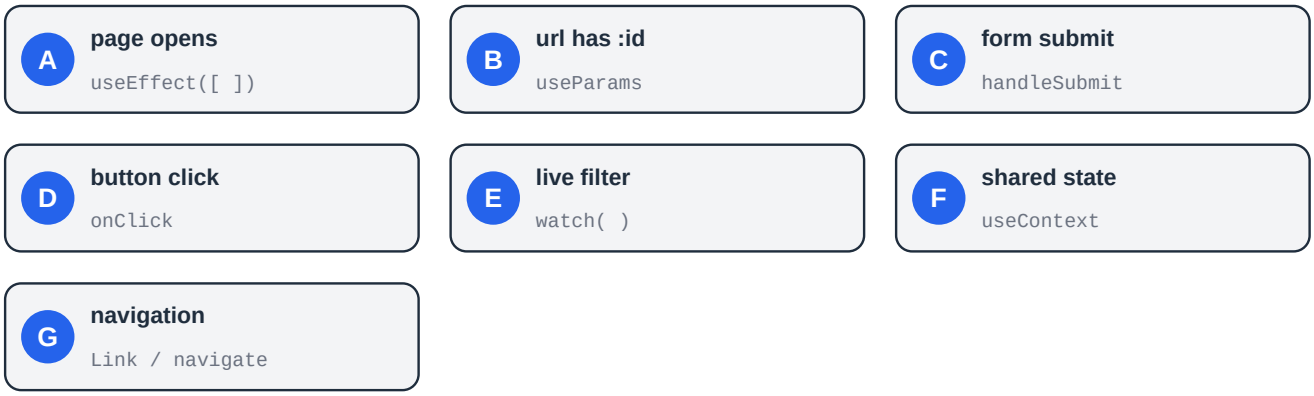
Every operation - view, add, edit, delete, login - is this same loop. Learn it once. Say it as: trigger -> call -> axios -> SQL -> json_encode -> setState.

Diagram 3 - the four CRUD operations side by side

	trigger	axios	SQL in PHP	updates screen
READ	`useEffect [ ]`	`axios.get`	`SELECT`	`setState(data)`
CREATE	`onSubmit`	`axios.post`	`INSERT`	`setServerMessage`
UPDATE	`onClick / submit`	`axios.post`	`UPDATE`	`setState(map)`
DELETE	`onClick`	`axios.post`	`DELETE`	`setState(filter)`

Only the trigger, the axios method, and the SQL word change between operations. READ uses GET; the other three use POST.

Diagram 4 - the seven kinds of trigger



Any 'what makes this run?' question is one of these seven. Name the letter, then point at the line.